Advanced Web Technologies

Building a Task API

Session 2

Dr. Zaher Yamak

Lebanese University Master M1 IN432



A brief recap of Session 1

Session 1 introduced requests that retrieve data with Flask.

A method and a path select a route. The route runs a Python function.

The response communicates a result through its HTTP status and its body.

Our next step is to accept input and change task data safely.

Lab review: what did your request prove?

Use one request from your previous lab.

1. Which method and path did you send?
2. Which status and body did you receive?
3. Which Python function produced that response?

Compare your evidence with a neighbour. Be ready to explain one line of code.

Lab review: a task response and an error

Request	Expected result in the task example	What it tells us
GET /api/health	200 and a health JSON object	The route responds
GET /api/tasks/1	200 and task 1	The task exists
GET /api/tasks/999	404 and an error	The target task is absent

A running application can correctly return 404 for an absent task.

Our scenario: a project team’s task list

Fictional teaching scenario: Lina uses a task list to prepare a project presentation.

She can read the list. Now she wants to add “Prepare project proposal”.

Later, she marks that task complete and tries an incorrect update.

Finally, the team removes the task. We follow the same task throughout.

Our task has three fields: id, title and done.

The session follows the task’s journey

Elapsed time	Focus
0–10 min	Brief recap and lab review
10–90 min	Create a task: input, validation, response and guided requests
90–105 min	Break
105–135 min	Update, delete and understand the starter’s modules
135–255 min	Laptop tutorial and request evidence
255–270 min	Review and the next storage problem

Lina’s new task targets the collection

POST /api/tasks asks the application to create a member of the task collection.

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Prepare project proposal"
}
```

The client supplies the title.

The server assigns the ID.

done starts as false when the client omits it.

This payload follows the task API’s creation contract.

Creation returns a task and its address

A fresh starter contains task IDs 1, 2 and 3. The first successful creation receives ID 4.

Lina’s request body

```
{
  "title": "Prepare project proposal"
}
```

POST /api/tasks

The created task

```
{
  "id": 4,
  "title": "Prepare project proposal",
  "done": false
}
```

201 and Location: /api/tasks/4

Flask parses the JSON body

request.get_json() reads the request body. A JSON object becomes a Python dictionary.

```
payload = request.get_json()

# Access the parsed title:
value = payload["title"]
# value is "Prepare project proposal"
```

payload now holds Python values.

Parsing checks whether the JSON text can be read.

Task rules still need checking.

A readable body is only the first step toward accepting a new task.

Prediction: a numeric title

Lina accidentally sends:

```
POST /api/tasks
Content-Type: application/json
{"title": 42}
```

Can Flask parse this JSON?
Should the application create the task?

Think individually for 30 seconds, then compare with a neighbour.

Parsing and task validation

Example input	Stage that decides	Result in this API
application/json with {"title":42}	Field validation	400, no task added
application/json with ["Prepare lab"]	Body shape validation	400, no task added
application/json with malformed text	JSON parsing	400, no task added
text/plain with valid JSON	Media type check	415, no task added

Answer: a numeric title parses, then fails the task’s title rule.

The creation input contract

Input	Rule	Reason in this exercise
Body	JSON object	Contains named fields
title	Required string, trim, 3–120 chars	The expected task description
done	Optional JSON boolean	Default false at creation
id	Reject client input	The server assigns identity
Other fields	Reject unknown names	An explicit, predictable contract

The same supplied-field rules will also apply when Lina updates a task.

Normalization prepares accepted fields

If Lina adds outer spaces, the validator trims them before checking the title’s length.

Parsed input: payload

```
{
  "title":
    "  Prepare project proposal  "
}
```

The validator starts with clean = {} and problems = {}.

Accepted fields: clean

```
{
  "title": "Prepare project proposal"
}
```

clean holds accepted values.
problems holds field feedback.

Title validation in Python

Excerpt from validate_task, executed when the payload contains title.

```
value = payload["title"]
if not isinstance(value, str):
    problems["title"] = "Title must be a string"
else:
    value = value.strip()
    if not 3 <= len(value) <= 120:
        problems["title"] = "Use 3 to 120 chars"
    else:
        clean["title"] = value
```

An empty title after trimming fails the length check.

done accepts a boolean, including false

JSON false becomes Python False. The string "false" remains a string.

```
if "done" in payload:
    if type(payload["done"]) is not bool:
        problems["done"] = "Done must be a JSON boolean"
    else:
        clean["done"] = payload["done"]
```

The presence check accepts a supplied False. Strings, numbers and null fail the type check.

On creation, an omitted done gets its default from TaskStore.

The validator makes one final decision

The field checks build clean and problems. Any recorded problem rejects the whole input.

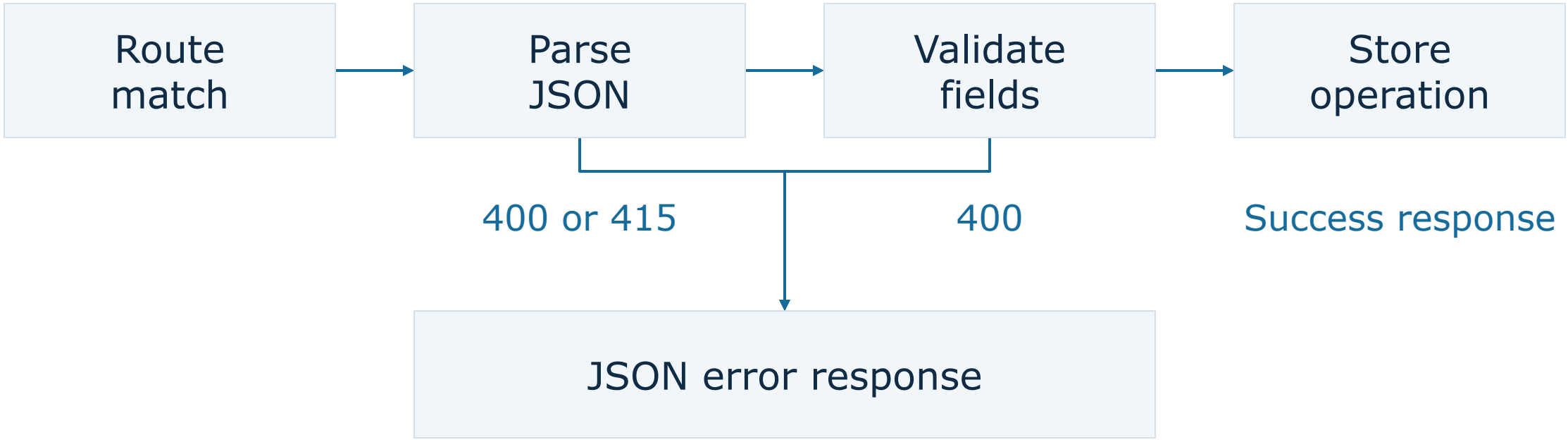
```
# Both dictionaries start empty:
clean = {}
problems = {}

# After all the field checks:
if problems:
    raise ApiError(
        400, "validation_error",
        "Invalid task data", problems)
return clean
```

The route receives clean only when problems remains empty.

Creation follows parsing and validation

A valid POST follows the top path. A failed check produces an error response.



The validator either returns clean or raises an error before the store call.

An error identifies the input to correct

For the numeric title, our API returns HTTP 400 and a structured JSON error.

```
{
  "error": {
    "code": "validation_error",
    "message": "Invalid task data",
    "fields": {
      "title": "Title must be a string"
    }
  }
}
```

The status reports rejection.

fields identifies the input to correct.

The supplied ApiError handler builds this body.

ApiError carries the status and feedback when validation raises it.

The POST handler coordinates creation

bp is the starter’s route group. Its /api/tasks prefix makes this collection route POST /api/tasks.

```
@bp.post("")
def create_task():
    payload = request.get_json()
    fields = validate_task(payload)
    item = store().create(fields)
```

Read the JSON body.

Receive clean fields after validation.

TaskStore assigns the ID and constructs the task.

store() is the supplied helper that retrieves this application’s TaskStore.

POST returns the new task’s Location

These lines continue create_task after the store has returned item.

```
    response = jsonify(item)
    response.status_code = 201
    response.headers["Location"] = url_for(
        "tasks.get_task", task_id=item["id"])
    return response
```

url_for uses the named GET endpoint and the new ID to build the item path.

For our first creation: status 201, the task body and Location /api/tasks/4.

Guided check: retrieve Lina’s new task

The server runs in one terminal. The client sends requests from a second terminal.

```
python api_client.py POST /api/tasks
  --json payloads/create.json

# Follow the returned Location. In a fresh app:
python api_client.py GET /api/tasks/4
```

Identify the assigned ID, the 201 status and Location. The following GET returns the created task.

The POST command wraps for display. Enter it on one line.

The new task now exists

15-minute break

After the break: Lina changes and removes the same task.

Lina marks task 4 complete

Our PATCH contract changes only the writable fields supplied in the JSON object.

```
PATCH /api/tasks/4
Content-Type: application/json

{
  "done": true
}
```

The path selects task 4.

The body supplies done.

The omitted title keeps its current value.

This API uses a JSON object of field values as its PATCH document.

PATCH keeps the omitted fields

Request: PATCH /api/tasks/4 with {"done": true}.

Before

```
{
  "id": 4,
  "title": "Prepare project proposal",
  "done": false
}
```

After

```
{
  "id": 4,
  "title": "Prepare project proposal",
  "done": true
}
```

The response contains the whole updated task, including fields absent from the request.

Partial validation handles omissions

Input decision	POST	PATCH with partial=True
title omitted	Reject: a new task needs a title	Keep the existing title
done omitted	The store defaults to false	Keep the existing done value
A field supplied	Validate its type and value	Validate its type and value
Empty object	Reject: title is missing	Reject: no change supplied

partial=True permits omissions. Every supplied field still has to pass its checks.

PATCH checks the target before the update

Our handler checks existence first, then parses and validates before the store update.

```
@bp.patch("/<int:task_id>")
def update_task(task_id):
    if store().get(task_id) is None:
        raise ApiError(404, "not_found", "Task not found")
    payload = request.get_json()
    fields = validate_task(payload, partial=True)
    item = store().update(task_id, fields)
    return jsonify(item)
```

An absent task returns 404. A valid update returns the task with HTTP 200.

Prediction: a partly valid PATCH

Task 4 is now complete. Lina sends:

PATCH /api/tasks/4
{"title": "Should not be saved", "done": "yes"}

What status should the response use?
What should a later GET show?

Think individually for 30 seconds, then compare with a neighbour.

The rejected PATCH leaves task 4 unchanged

Answer: HTTP 400. A later GET still returns the previous task representation.

```
{
  "id": 4,
  "title": "Prepare project proposal",
  "done": true
}
```

The done check fails.

The validator raises before returning clean.

The handler never reaches store().update().

The valid title does not reach storage when another supplied field fails.

The team no longer needs task 4

Request	Expected result	Stored task 4
DELETE /api/tasks/4	204 with an empty body	Absent
GET /api/tasks/4 afterwards	404 with a JSON error	Still absent

A client handles 204 without trying to parse a JSON body.

DELETE chooses its response from the store

TaskStore.delete returns True for a removed item and False for an absent item.

```
@bp.delete("/<int:task_id>")
def delete_task(task_id):
    if not store().delete(task_id):
        raise ApiError(404, "not_found",
                       "Task not found")
    return "", 204
```

The ID selects the task.

A False result raises our 404 error.

The empty string supplies no content on success.

This DELETE contract needs no JSON request body.

Prediction: Lina repeats the deletion

The first DELETE /api/tasks/4 returns 204.
The second returns 404.

Does the second request change the final result?

What could happen if Lina repeated the original POST instead?

Think individually for 30 seconds, then compare with a neighbour.

Idempotence describes the intended effect

Repeating the operation has the same intended effect.

Both DELETE attempts leave task 4 absent, even though their statuses differ.

Repeated successful POST requests can create additional task IDs.

A PATCH that sets done to true is idempotent. Other PATCH operations can behave differently.

The API has different responsibilities

Our handlers coordinate work, but input rules and task storage have distinct responsibilities.

A title rule should stay consistent in both POST and PATCH.

A storage change should leave the published request and response contract recognizable.

The starter separates these responsibilities into modules within one Flask application.

Modularity makes those responsibilities easier to locate and change.

Each starter module has a focused role

Module	Responsibility	Core student edit
tasks.py	Routes, operation order and responses	POST, PATCH, DELETE
validation.py	Accepted fields and input rules	validate_task
store.py	Task IDs and in-memory operations	Provided
errors.py	Consistent JSON error responses	Provided
__init__.py	Build the app and register its parts	Provided

The core lab changes two files: validation.py and tasks.py.